Домашнее задание №3: Аналитика IT-вакансий на объектном хранилище

Коханенко Вадим Вячеславович, группа J3310. 17.10.2026

Задача

Вариант 6: **«Программист, разработчик на C и C++»**. Поднять RustFS, загрузить в него датасет вакансий, посчитать частоты навыков, пар и троек навыков двумя независимыми способами (свой MapReduce и SQL в DuckDB), сверить результаты, оценить влияние навыков на медианную зарплату, сравнить три региона и снять профиль по времени и памяти. Бонус: оптимизировать расчёт и показать «до» и «после».

Структура репозитория (папка `HW3`):

Путь	Что там
`docker-compose.yml`, `RUN.md`	RustFS на 4 папках-дисках (тот же стенд, что в HW2) и порядок запуска
`src/`	весь код (все скрипты запускаются из корня `HW3`)
`report/`	этот отчёт (`report.md`, `report.pdf`) и скриншоты `images/`
`results/`	результаты расчётов, выгруженные из хранилища, профилирование и консольные логи

Скрипты из `src/`:

Файл	Что делает
`gen_data.py`	генератор датасета вакансий
`upload.py`, `ls_bucket.py`	создание бакета `hh-lake`, загрузка данных, список объектов
`mapreduce.py`	свой MapReduce (финальная версия)
`mapreduce_v1.py`	первая версия MapReduce, до оптимизации (для бонуса)
`duckdb_sql.py`	тот же расчёт SQL-запросом в DuckDB
`verify.py`	построчное сравнение результатов двух способов
`analysis.py`	этапы 5 и 6: зарплаты и регионы
`profile.py`	профилирование (3 запуска, время и пик памяти)
`export_results.py`, `make_charts.py`	выгрузка результатов из хранилища в `results/`, графики для отчёта
`common.py`	общие настройки и функции

1. Развёртывание хранилища

RustFS поднимается так же, как в HW2: 4 папки-диска, erasure set 2+2. Параметры доступа: пользователь `antrall`, S3 API на порту 9000.

```
docker compose up -d
```

Проверка, что сервис живой:

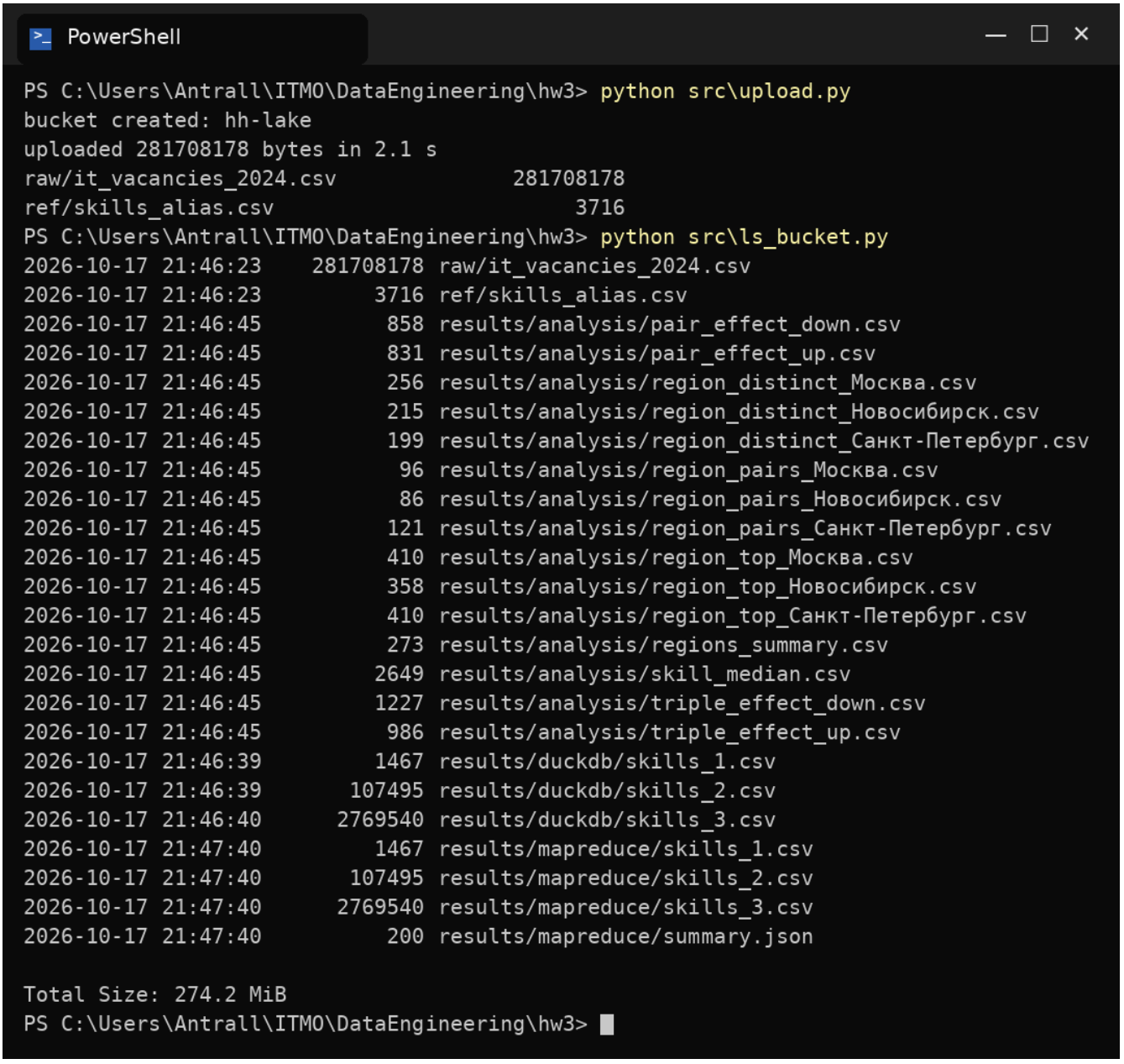


2. Данные и загрузка

Настоящую выгрузку hh.ru мне собрать не удалось, поэтому датасет сделан генератором `gen_data.py`. Структура файла как у выгрузки из HW2 (те же 23 колонки: `vacancy_id`, `area`, `role`, `salary_from`, `salary_to`, `currency`, `gross`, `key_skills`, `description` и т. д.).

- всего **250 000** вакансий 11 сегментов (Python, Java, Go, 1C, фронтенд, аналитик, DevOps, PHP, мобильная разработка, QA и C/C++), файл 281 708 178 байт (≈ 269 МиБ);
- вакансий моего варианта (`role = «Программист, разработчик на C и C++»`) **49 801**. Остальные сегменты нужны, чтобы фильтр по варианту реально что-то отсекал: в них иногда встречаются C++, Linux и Git, и если фильтр забыть, счётчики изменятся;
- навыки записаны так же грязно, как на hh.ru: `C++`, `c++`, кириллическая `C++`, `Си++`, `Linux` / `линукс`, `Git` / `GIT`, встречаются дубли и пустое поле. Для нормализации сделан словарь синонимов `skills_alias.csv` (приведение к нижнему регистру, `trim`, поиск по словарю; навыков, которых в словаре нет, оставляются «как есть»).

Загрузка: бакет `hh-lake`, файл идёт через multipart по 8 МиБ, словарь синонимов лежит в `ref/`.



В бакете три «каталога»: `raw/` (исходные данные), `ref/` (словарь) и `results/` (результаты обоих способов, раздел 4).

3. Расчёт навыков двумя способами

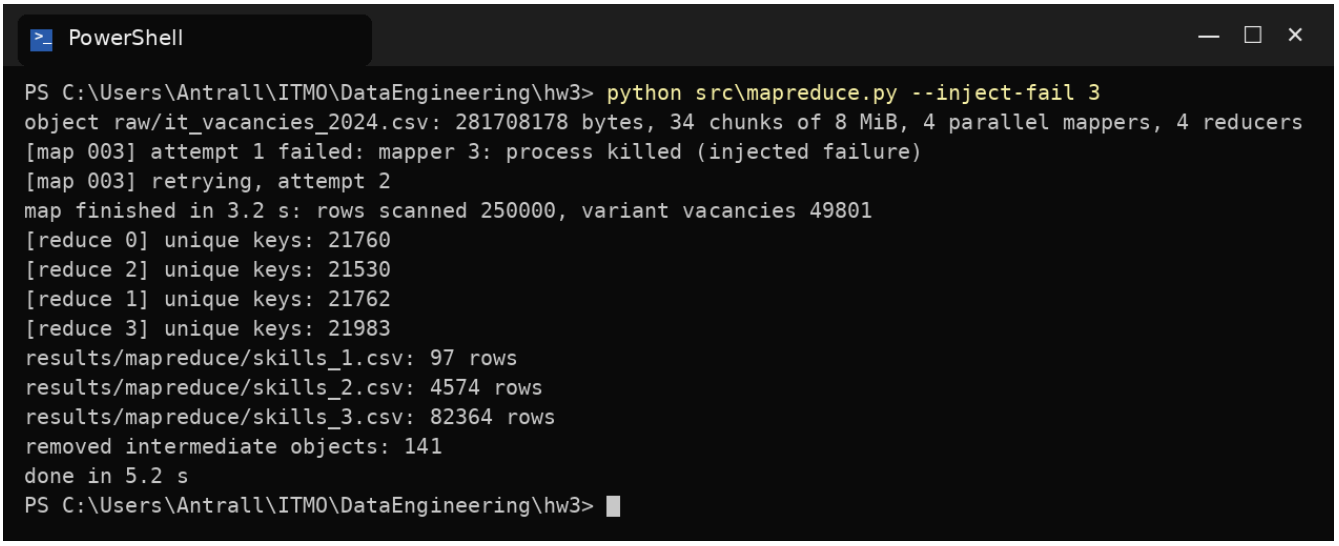
Что считаем: для каждой вакансии варианта берётся **множество** нормализованных навыков (один навык, записанный дважды, считается один раз), и для него считаются все подмножества размера 1, 2 и 3. Частота = в скольких вакансиях встретилось.

3.1 Свой MapReduce (`mapreduce.py`)

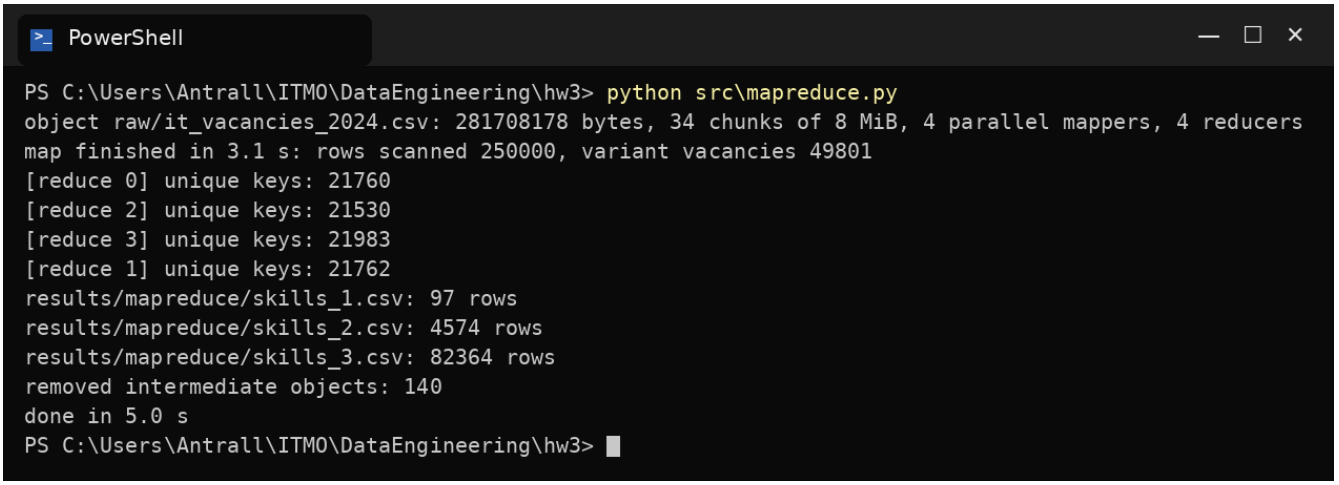
- **Чанки.** Файл не скачивается целиком. Мастер режет объект на чанки по 8 МиБ (получилось 34), каждый маппер забирает свой кусок Range-запросом (`bytes=start-end`) прямо из хранилища.
- **Границы строк.** Граница чанка почти всегда попадает в середину строки. Правило как в Hadoop: чанк пропускает начало строки, которая началась в

предыдущем чанке, а свою последнюю строку дочитывает за границей (для этого запрашивается запас 64 КиБ). Проверка: маппера суммарно увидели **250 000** строк, ровно столько, сколько в файле, ни потерь, ни дублей. В описании вакансий нет переводов строк (в генераторе они заменены пробелами), поэтому деление по `\n` безопасно.

- **Map.** Фильтр по роли, нормализация навыков, `combinations` по отсортированному списку навыков для k = 1, 2, 3, локальное накопление (combiner) и раскладка ключей по 4 партициям (`crc32(ключ) % 4`). Промежуточные файлы маппера пишутся обратно в S3 (`tmp/…`).
- **Reduce.** 4 редьюсера, каждый читает свою партицию от всех мапперов, суммирует и пишет результат в S3. Мастер склеивает партиции в три итоговых файла `skills_1.csv` , `skills_2.csv` , `skills_3.csv` , отсортированных по убыванию частоты.
- **Отказ маппера.** Флаг `--inject-fail 3` заставляет маппер чанка 3 упасть в первой попытке уже после того, как он записал часть промежуточных файлов. Мастер перезапускает задачу, а редьюсеры берут только файлы «зафиксированной» (успешной) попытки, поэтому недописанный мусор упавшей попытки в результат не попадает. После завершения промежуточные объекты удаляются (141 объект, включая «осиротевшие» от упавшей попытки).



После этого запуска результаты совпали с DuckDB (см. раздел 5), то есть перезапуск ничего не испортил. Обычный запуск без отказа:



3.2 DuckDB (`duckdb_sql.py`)

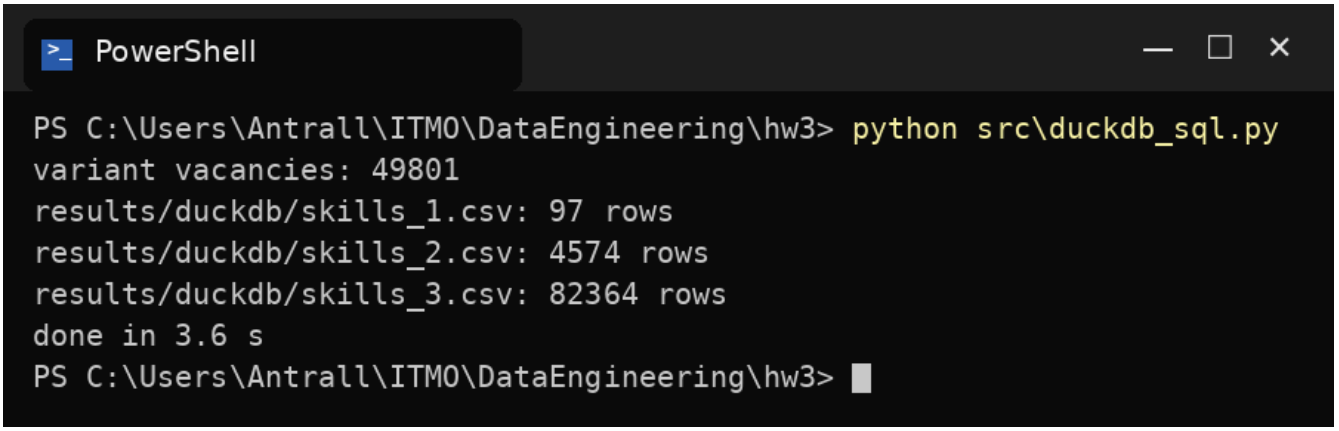
Запросы пишутся к тем же данным в хранилище. Расширение `httpfs` у меня не поставилось (нет доступа к `extensions.duckdb.org`), поэтому S3 подключён через `s3fs` и `register_filesystem`. Файл читается из хранилища, не с локального диска.

Главные запросы (полный текст в `duckdb_sql.py`):

```
-- пары (вакансия, навык) после нормализации; DISTINCT, чтобы дубль навыка в вакансии не
считался дважды
CREATE TABLE vs AS
SELECT DISTINCT v.vacancy_id, COALESCE(a.skill, v.s) AS skill
FROM (SELECT vacancy_id, lower(trim(unnest(string_split(key_skills, ',')))) AS s FROM
vac) v
LEFT JOIN alias a ON a.alias = v.s
WHERE v.s <> '';

-- пары
SELECT a.skill, b.skill, count(*) AS vacancies
FROM vs a JOIN vs b ON a.vacancy_id = b.vacancy_id AND a.skill < b.skill
GROUP BY 1, 2 ORDER BY vacancies DESC, 1, 2;
```

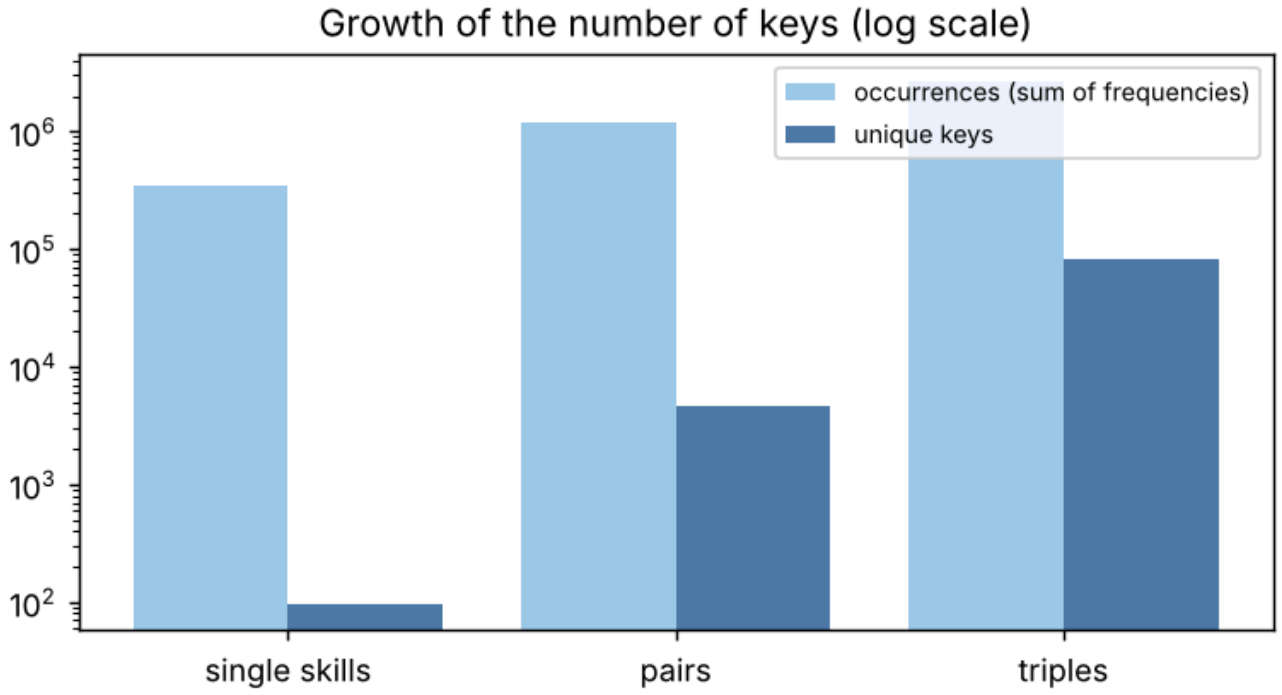
Тройки получаются таким же самосоединением в три таблицы с условием `a.skill < b.skill AND b.skill < c.skill`. Условие `<` нужно, чтобы каждая комбинация считалась один раз, а не в 2 или 6 перестановках.



4. Результаты для трёх размеров стека

Результаты обоих способов лежат в хранилище: `results/mapreduce/skills_{1,2,3}.csv` и `results/duckdb/skills_{1,2,3}.csv`. Копии выгружены в папку `results/` репозитория (`export_results.py`).

Размер стека	Уникальных комбинаций	Всего вхождений (сумма частот)	Вхождений на вакансию
1 навык	97	344 000	6.9
пара	4 574	1 195 175	24.0
тройка	82 364	2 696 570	54.1



Почему число комбинаций так быстро растёт. В вакансии с n навыками получается C(n, 1) = n навыков, C(n, 2) = n(n−1)/2 пар и C(n, 3) = n(n−1)(n−2)/6 троек. При среднем n ≈ 6.9 это ≈ 7, ≈ 21 и ≈ 35 комбинаций, а в таблице 6.9, 24.0 и 54.1: комбинаций получается больше, чем по средней вакансии, потому что C(n, k) растёт быстрее линейной функции, и вакансии с 12–15 навыками дают непропорционально много пар и троек. Уникальных пар в 47 раз больше, чем навыков (97 → 4 574), а уникальных троек в 18 раз больше, чем пар. Рост замедляется, потому что реальных сочетаний меньше возможных: из 97 навыков теоретически C(97, 2) = 4 656 пар (встретились 4 574, почти все) и C(97, 3) = 147 440 троек (встретилась 82 364, чуть больше половины), т. е. навыки в вакансии не независимы и редкие тройки просто не успевают набраться.

Самые частые одиночные навыки: C++ (23 734 вакансий), Git (16 404), Linux (12 536), STL (8 283), CMake (6 925). Самые частые пары: C++ + Git, C++ + Linux.

5. Верификация

`verify.py` сравнивает файлы обоих способов построчно, в том числе порядок строк и значения частот (не только суммы):

```
PowerShell
PS C:\Users\Antrall\ITMO\DataEngineering\hw3> python src\verify.py
skills : results/mapreduce=97 rows, results/duckdb=97 rows, frequency sum 344000 / 344000 -> MATCH
pairs  : results/mapreduce=4574 rows, results/duckdb=4574 rows, frequency sum 1195175 / 1195175 -> MATCH
triples: results/mapreduce=82364 rows, results/duckdb=82364 rows, frequency sum 2696570 / 2696570 -> MATCH
RESULT: results match completely
PS C:\Users\Antrall\ITMO\DataEngineering\hw3>
```

Все три размера стека совпали полностью: 97, 4 574 и 82 364 строки, суммы частот 344 000 / 1 195 175 / 2 696 570 одинаковы. Совпадение есть и после запуска с перезапуском упавшего маппера.

6. Влияние навыков на зарплату

Из 49 801 вакансий зарплата указана в 28 681 (57.6%), только они участвуют в этом этапе. Приведение к одной шкале: если указана вилка, берётся её середина; одна граница берётся как есть; валюта пересчитана в рубли (USD = 90, EUR = 100, KZT = 0.19); если зарплата указана до вычета НДФЛ (`gross = true`), умножаю на 0.87, то есть везде получается «на руки». **Медиана по всем вакансиям с зарплатой: 147 030 руб.**

Прибавку считаю относительно «базы», а не только относительно общей медианы, чтобы убрать эффект самого популярного навыка (например, у C++ медиана почти равна общей):

- для пары: медиана вакансий с этой парой − медиана вакансий с более частым навыком из пары;
- для тройки: медиана тройки − медиана пары из двух остальных, более частых, навыков (то есть прибавка от добавления третьего, самого редкого);
- порог: не менее 100 вакансий с зарплатой для пары и не менее 60 для тройки, чтобы медиана не прыгала на малой выборке.

6.1 Пары, которые повышают медианную зарплату

Пара навыков	вакансий с зарплатой	медиана пары	базовый (более частый) навык	медиана базового	прибавка к базовому	отличие от общей медианы
C++ + Low latency	1323	357,500	C++	152,250	205,250	210,470
Low latency + Алгоритмы и структуры данных	361	355,830	Алгоритмы и структуры данных	152,250	203,580	208,800
Linux + Low latency	1046	372,360	Linux	169,000	203,360	225,330
Low latency + Python	449	376,000	Python	173,130	202,870	228,970
CMake + Low latency	439	368,010	CMake	169,500	198,510	220,980
Low latency + STL	426	350,500	STL	160,515	189,985	203,470
Git + Low latency	832	341,000	Git	154,500	186,500	193,970
Low latency + TCP/IP	460	372,000	TCP/IP	194,880	177,120	224,970
Boost + Low latency	370	357,275	Boost	180,960	176,315	210,245
C++17 + SIMD	215	387,000	C++17	212,000	175,000	239,970

6.2 Пары, которые понижают медианную зарплату

Пара навыков	вакансий с зарплатой	медиана пары	базовый (более частый) навык	медиана базового	прибавка к базовому	отличие от общей медианы
C++20 + Redis	105	161,820	C++20	239,000	-77,180	14,790
Delphi + Алгоритмы и структуры данных	122	77,430	Алгоритмы и структуры данных	152,250	-74,820	-69,600
C++20 + Protobuf	123	164,430	C++20	239,000	-74,570	17,400
C++20 + CI/CD	162	166,302	C++20	239,000	-72,698	19,272
C++ + Delphi	738	82,250	C++	152,250	-70,000	-64,780
C++ + MFC	985	84,390	C++	152,250	-67,860	-62,640
MFC + Алгоритмы и структуры данных	131	84,500	Алгоритмы и структуры данных	152,250	-67,750	-62,530
C++ + Pascal	512	86,000	C++	152,250	-66,250	-61,030
C++ + Visual Studio	973	88,740	C++	152,250	-63,510	-58,290
C++ + SVN	859	92,000	C++	152,250	-60,250	-55,030

6.3 Тройки, которые повышают медианную зарплату

Тройка навыков	вакансий с зарплатой	медиана тройки	базовая пара	медиана пары	прибавка к паре	отличие от общей медианы
CMake + Low latency + Python	152	395,555	CMake + Python	185,745	209,810	248,525
C++ + Low latency + Python	440	383,670	C++ + Python	177,480	206,190	236,640
CMake + Low latency + STL	148	373,665	CMake + STL	171,390	202,275	226,635
C++ + Linux + Low latency	1024	377,624	C++ + Linux	176,000	201,624	230,594
C++17 + Python + SIMD	70	419,670	C++17 + Python	219,240	200,430	272,640
CMake + Linux + Low latency	358	380,595	CMake + Linux	180,525	200,070	233,565
Bash + Low latency + Python	86	418,905	Bash + Python	219,555	199,350	271,875
C++ + Low latency + Алгоритмы и структуры данных	358	357,350	C++ + Алгоритмы и структуры данных	158,340	199,010	210,320
C++ + CMake + Low latency	431	371,500	C++ + CMake	174,000	197,500	224,470

Тройка навыков	вакансий с зарплатой	медиана тройки	базовая пара	медиана пары	прибавка к паре	отличие от общей медианы
C++17 + Low latency + Python	160	416,686	C++17 + Python	219,240	197,446	269,656

6.4 Тройки, которые понижают медианную зарплату

Тройка навыков	вакансий с зарплатой	медиана тройки	базовая пара	медиана пары	прибавка к паре	отличие от общей медианы
C++20 + Linux + Redis	86	158,018	C++20 + Linux	250,560	-92,542	10,988
C++20 + Protobuf + Многопоточность	75	164,430	C++20 + Многопоточность	253,605	-89,175	17,400
C++20 + CI/CD + Многопоточность	92	164,900	C++20 + Многопоточность	253,605	-88,705	17,870
Linux + Алгоритмы и структуры данных + ООП	169	97,000	Linux + Алгоритмы и структуры данных	182,700	-85,700	-50,030
Asio + C++20 + Linux	73	165,000	C++20 + Linux	250,560	-85,560	17,970
C++20 + Linux + Protobuf	105	165,300	C++20 + Linux	250,560	-85,260	18,270
C++20 + CI/CD + TCP/IP	83	161,000	C++20 + TCP/IP	244,250	-83,250	13,970
Python + Алгоритмы и структуры данных + ООП	102	96,035	Python + Алгоритмы и структуры данных	179,000	-82,965	-50,995
C++ + Delphi + Алгоритмы и структуры данных	103	75,500	C++ + Алгоритмы и структуры данных	158,340	-82,840	-71,530
C++ + C++20 + Redis	103	161,820	C++ + C++20	243,800	-81,980	14,790

6.5 Одиночные навыки для контекста (не менее 300 вакансий с зарплатой)

Выше всего:

Навык	вакансий с зарплатой	медиана, руб	отличие от общей медианы
Low latency	1370	348,500	201,470
Алготрейдинг	575	281,010	133,980
C++20	1911	239,000	91,970
SIMD	1112	238,908	91,878
Технический английский	1184	233,548	86,518
FIX protocol	493	218,000	70,970

Навык	вакансий с зарплатой	медиана, руб	отличие от общей медианы
C++17	2517	212,000	64,970
Rust	432	206,250	59,220
Bash	1550	206,190	59,160
CUDA	882	200,970	53,940

Ниже всего:

Навык	вакансий с зарплатой	медиана, руб	отличие от общей медианы
C	6570	122,670	-24,360
Windows	4106	117,450	-29,580
WinAPI	1809	109,000	-38,030
ООП	3328	103,000	-44,030
Access	383	99,180	-47,850
SVN	1015	90,000	-57,030
Visual Studio	1202	87,250	-59,780
Pascal	619	84,390	-62,640
Delphi	907	82,650	-64,380
MFC	1172	82,000	-65,030

6.6 Итог: что повышает и что понижает

Повышают зарплату:

- **Low latency** (медиана 348 500 руб, +201 тыс. к общей). Все 10 лучших пар и большая часть лучших троек содержат именно его; остальные навыки в паре почти не важны, просто пара наследует эффект Low latency (+175…205 тыс. к медиане базового навыка). Это вакансии торговых систем и систем реального времени, там платят вдвое-втрое больше рынка;
- **Алготрейдинг** (281 010) и **FIX protocol** (218 000): та же область;
- **SIMD** (238 908), **C++20** (239 000), **C++17** (212 000): современные стандарты и оптимизация под процессор просят в вакансиях, где важна производительность. В парах и тройках без Low latency лучшие из них: C++17 + SIMD (медиана 387 000), C++17 + Python + SIMD (419 670);
- **Rust**, **CUDA**, **Bash**, **TCP/IP**, **Технический английский** (+48…87 тыс. к общей медиане): редкие и «системные» навыки, их ищут для узких задач.

Понижают зарплату:

- **MFC (82 000), Delphi (82 650), Pascal (84 390), Visual Studio (87 250), SVN (90 000), Access (99 180), WinAPI (109 000)**: поддержка старых десктопных систем. Такие вакансии платят в 1.6–1.8 раза меньше общей медианы. Пары вида «C++ + Delphi» (−70 000 к базовому C++), «C++ + MFC» (−67 860) и «C++ + Pascal» (−66 250) падают сильнее всего, потому что добавляют к C++ устаревший стек;
- **ООП (103 000)** и **Алгоритмы и структуры данных** в тройках («Linux + Алгоритмы + ООП» −85 700, «Python + Алгоритмы + ООП» −82 965): это

формулировки из вакансий для начинающих, где перечисляют базовые знания вместо конкретных технологий;
- **Windows (117 450)** и **C (122 670)**: общие требования, часто стоят в вакансиях низкого уровня оплаты.

Особый случай: «C++20 + Redis», «C++20 + Protobuf», «C++20 + CI/CD» понижают на 70–77 тыс. Это не значит, что Redis или Protobuf обесценивают C++20. Базовый навык C++20 имеет высокую медиану (239 000) потому, что часто встречается в вакансиях торговых систем. Redis, Protobuf и CI/CD характерны для обычного серверного бэкенда, где платят меньше. Если сравнивать с **общей** медианой, у этих пар прибавка положительная (+15…19 тыс.). То есть здесь «падение» вызвано сменой типа вакансии (состав выборки), а не самим навыком. Именно поэтому я привожу обе колонки: «прибавка к базовому» и «отличие от общей медианы».

7. Региональный анализ

Выбрал три региона с разным профилем рынка: **Москва**, **Санкт-Петербург** и **Новосибирск**. Индекс востребованности = доля навыка в вакансиях региона / доля навыка во всех вакансиях варианта (больше 1 — в регионе навык ищут чаще среднего).

7.1 Сводка по регионам

Регион	вакансий	с зарплатой	% с зарплатой	медиана, руб	p25	p75	% удалённой работы
Москва	16881	9728	57.6	194,880	132,000	290,000	17.4
Санкт-Петербург	8592	4946	57.6	159,210	110,014	232,669	18.3
Новосибирск	2760	1562	56.6	121,000	85,260	171,956	19

Медиана в Москве в 1.22 раза выше, чем в Петербурге (194 880 против 159 210), и в 1.61 раза выше, чем в Новосибирске (121 000). Разброс тоже больше: межквартильный размах в Москве 158 тыс., в Новосибирске 87 тыс. Доля вакансий с указанной зарплатой (≈ 57%) и доля удалённой работы (17–19%) в регионах почти одинаковы, разница между регионами в самих зарплатах и в наборе навыков.

7.2 Москва

Топ-10 навыков по числу вакансий:

Навык	вакансий	доля в регионе, %	индекс востребованности	медиана зарплаты, руб
C++	14450	85.6	1.03	200,970
Git	9800	58.1	1.02	202,710
Linux	8189	48.5	1.1	225,500
STL	5324	31.5	1.1	206,095
CMake	4462	26.4	1.1	219,240
Многопоточность	4166	24.7	1.22	246,105

Навык	вакансий	доля в регионе, %	индекс востребованности	медиана зарплаты, руб
Python	4020	23.8	1.07	227,070
C	3041	18	0.79	155,000
TCP/IP	2919	17.3	1.25	246,105
Алгоритмы и структуры данных	2907	17.2	1.12	203,145

Навыки, которых в Москве больше среднего (не менее 100 вакансий):

Навык	вакансий	доля в регионе, %	индекс востребованности	медиана зарплаты, руб
Low latency	1232	7.3	1.51	424,250
Алготрейдинг	474	2.8	1.42	351,250
C++20	1510	8.9	1.36	314,940
Технический английский	960	5.7	1.34	306,120
C++17	1955	11.6	1.31	274,050

Топ-5 пар: C++ + Git (8774), C++ + Linux (7445), Git + Linux (5236), C++ + STL (5033), C++ + CMake (4139).

В Москве выделяются торговые системы: **Low latency** встречается в 7.3% вакансий (индекс 1.51), медиана по ним 424 250 руб, **Алготрейдинг** (индекс 1.42, медиана 351 250). Из-за них и высокая общая медиана и широкий разброс.

7.3 Санкт-Петербург

Навык	вакансий	доля в регионе, %	индекс востребованности	медиана зарплаты, руб
C++	7199	83.8	1.01	163,500
Git	4932	57.4	1.01	168,000
Linux	3334	38.8	0.88	179,000
STL	2469	28.7	1	168,172
Python	1882	21.9	0.99	180,090
CMake	1863	21.7	0.9	180,308
Многопоточность	1773	20.6	1.02	197,962
C	1759	20.5	0.89	136,545
Windows	1399	16.3	1.14	136,590
Алгоритмы и структуры данных	1337	15.6	1.01	165,150

Навыки, которых в Петербурге больше среднего:

Навык	вакансий	доля в регионе, %	индекс востребованности	медиана зарплаты, руб
Unreal Engine	885	10.3	1.69	163,780
DirectX	512	6	1.64	170,520
Unity	307	3.6	1.5	164,430

Навык	вакансий	доля в регионе, %	индекс востребованности	медиана зарплаты, руб
Lua	280	3.3	1.49	173,130
Vulkan	303	3.5	1.47	175,500

Топ-5 пар: C++ + Git (4339), C++ + Linux (2930), C++ + STL (2348), Git + Linux (2143), C++ + Многопоточность (1728).

Петербург выделяется **геймдевом**: Unreal Engine (10.3% вакансий, индекс 1.69), DirectX (1.64), Unity (1.50), Vulkan, Lua. Зарплаты по этим навыкам (≈ 164–175 тыс.) лишь немного выше медианы региона (159 тыс.), то есть сильной премии за игровой стек нет.

7.4 Новосибирск

Навык	вакансий	доля в регионе, %	индекс востребованности	медиана зарплаты, руб
C++	2122	76.9	0.93	121,000
Git	1605	58.2	1.02	127,000
Linux	1177	42.6	0.97	138,330
C	1016	36.8	1.6	115,710
CMake	648	23.5	0.98	136,078
STL	626	22.7	0.79	127,010
Embedded	606	22	1.76	122,670
Python	570	20.7	0.93	145,290
Микроконтроллеры	442	16	1.83	130,500
RTOS	403	14.6	1.91	122,670

Навыки, которых в Новосибирске больше среднего:

Навык	вакансий	доля в регионе, %	индекс востребованности	медиана зарплаты, руб
RTOS	403	14.6	1.91	122,670
CAN	242	8.8	1.86	120,060
Микроконтроллеры	442	16	1.83	130,500
GDB	288	10.4	1.82	124,410
SPI/I2C/UART	333	12.1	1.78	124,410

Топ-5 пар: C++ + Git (1289), C++ + Linux (963), Git + Linux (749), C + C++ (604), C + Git (591).

Новосибирск выделяется **встраиваемыми системами**: RTOS (индекс 1.91), микроконтроллеры (1.83), Embedded (1.76), чистый C (индекс 1.60, у 36.8% вакансий). Эти навыки в целом оплачиваются ниже («Embedded» медиана 132 тыс. по всем регионам), в сочетании с низкой общей медианой региона получается самый низкий результат из трёх.

7.5 Наблюдения

1. **Одни и те же навыки стоят по-разному.** C++ в Москве: 200 970 руб, в Петербурге: 163 500, в Новосибирске: 121 000. Linux: 225 500 / 179 000 / 138 330. Разница между регионами (≈ 1.6 раза Москва/Новосибирск) почти не зависит от навыка, то есть региональный коэффициент есть сам по себе.
2. **Региональный профиль определяется отраслью:** Москва: финансы и торговые системы, Петербург: игры, Новосибирск: встраиваемые системы. Поэтому «топ-10 навыков» во всех трёх регионах почти одинаковый (C++, Git, Linux, STL), а отличаются навыки с индексом больше 1.4.
3. **Москва выигрывает не только из-за дороговизны региона, но и из-за состава вакансий:** именно здесь сосредоточены дорогие вакансии Low latency и алготрейдинга, они поднимают и медиану, и верхний квартиль.
4. **Для Новосибирска вывод с оговоркой:** выборка с зарплатой 1 562 вакансий, это в 6 раз меньше, чем в Москве. Тем не менее порог в 100 вакансий соблюдён для всех навыков в таблицах.

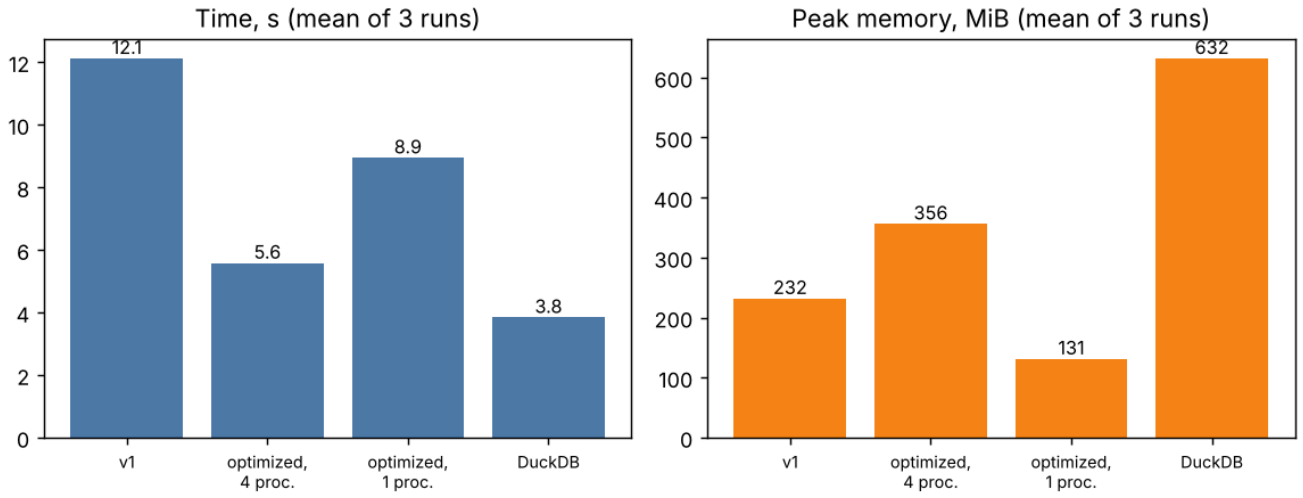
8. Профилирование и оптимизация

Профилирование: каждый способ запускается отдельным процессом 3 раза, время измеряется от старта до завершения (включая чтение из хранилища и запись результатов обратно), пиковая память = максимум суммы RSS процесса и всех его дочерних процессов (опрос каждые 20 мс, `psutil`). Скрипт: `profile.py`.

Способ	время, с (3 запуска)	среднее время, с	пик памяти, МиБ (3 запуска)	средняя память, МиБ
MapReduce v1 (до оптимизации)	11.7 / 12.2 / 12.4	**12.1**	234 / 232 / 229	**232**
MapReduce (оптимизированный, 4 процесса)	5.8 / 5.5 / 5.4	**5.6**	358 / 355 / 355	**356**
MapReduce (оптимизированный, 1 процесс)	8.7 / 8.6 / 9.5	**8.9**	131 / 131 / 131	**131**
DuckDB SQL	5.0 / 3.3 / 3.2	**3.8**	648 / 640 / 607	**632**

```
PowerShell
PS C:\Users\Antrall\ITMO\DataEngineering\hw3> python src\profile.py 3
MapReduce v1 (before optimization)   run 1:   11.7 s, peak memory     234 MiB
MapReduce v1 (before optimization)   run 2:   12.2 s, peak memory     232 MiB
MapReduce v1 (before optimization)   run 3:   12.4 s, peak memory     229 MiB
MapReduce (optimized, 4 processes)   run 1:    5.8 s, peak memory     358 MiB
MapReduce (optimized, 4 processes)   run 2:    5.5 s, peak memory     355 MiB
MapReduce (optimized, 4 processes)   run 3:    5.4 s, peak memory     355 MiB
MapReduce (optimized, 1 process)     run 1:    8.7 s, peak memory     131 MiB
MapReduce (optimized, 1 process)     run 2:    8.6 s, peak memory     131 MiB
MapReduce (optimized, 1 process)     run 3:    9.5 s, peak memory     131 MiB
DuckDB SQL                           run 1:    5.0 s, peak memory     648 MiB
DuckDB SQL                           run 2:    3.3 s, peak memory     640 MiB
DuckDB SQL                           run 3:    3.2 s, peak memory     607 MiB

average over runs:
MapReduce v1 (before optimization)   time   12.1 s (min 11.7, max 12.4), memory     232 MiB (max 234)
MapReduce (optimized, 4 processes)   time    5.6 s (min 5.4, max 5.8), memory     356 MiB (max 358)
MapReduce (optimized, 1 process)     time    8.9 s (min 8.6, max 9.5), memory     131 MiB (max 131)
DuckDB SQL                           time    3.8 s (min 3.2, max 5.0), memory     632 MiB (max 648)
PS C:\Users\Antrall\ITMO\DataEngineering\hw3>
```



8.1 Бонус: что оптимизировал в MapReduce

Первая версия (`mapreduce_v1.py`) работала в один процесс, читала файл чанками по 32 МиБ, разбирала каждую строку csv-парсером и писала в хранилище **по строке на каждое вхождение** навыка, пары или тройки (всего 4 235 745 строк в промежуточных файлах). Что изменено в `mapreduce.py`:

1. **Предфильтр по байтам.** Строка сначала проверяется на наличие названия роли (`role_b in line`), и только подходящие 20% идут в `csv`-парсер. Остальные 80% строк пропускаются без разбора.
2. **Кэш нормализации.** Каждое «сырое» написание навыка (`c++`, `C++`, ...) нормализуется один раз, а не при каждой встрече.
3. **Combiner.** Маппер сначала суммирует локально (`Counter`), в хранилище пишутся уже готовые пары «ключ → количество», а не повторяющиеся строки «ключ → 1». Это сильно сокращает промежуточные данные и работу редьюсеров.
4. **Параллелизм.** Мапперы и редьюсеры работают в пуле процессов (4 процесса), чанк уменьшен до 8 МиБ, чтобы нагрузка делилась равномернее.

Результат:

	время, с	память, МиБ
v1 (до)	12.1	232
оптимизированный, 4 процесса (после)	5.6 (в 2.2 раза быстрее)	356 (на 54% больше)
оптимизированный, 1 процесс	8.9 (на 26% быстрее)	131 (на 43% меньше)

Что видно. Параллелизм ускоряет расчёт в 2.2 раза, но платой становится память: 4 процесса держат по своему `Counter` и своему чанку, поэтому пик 356 МиБ против 232 у первой версии. Чтобы отделить эффект самих оптимизаций (1–3) от параллелизма, я запустил финальную версию с `--workers 1 --reducers 1`: она быстрее v1 на 26% и при этом потребляет на 43% меньше памяти (combiner и мелкие чанки вместо накопления всех вхождений в списке). То есть за скорость при 4 процессах заплачено памятью, и если памяти мало, лучше запускать меньше воркеров.

8.2 Сравнение с DuckDB

DuckDB быстрее всех (3.8 с, то есть быстрее оптимизированного MapReduce в 1.5 раза), но берёт больше всего памяти (632 МиБ): он материализует таблицы `vs` и промежуточные соединения для пар и троек (самосоединение даёт ≈ 2.7 млн строк для троек до агрегации). Он оптимизирован под такой «групповой» запрос, использует все ядра и векторизованное выполнение, в то время как мой MapReduce написан на чистом Python. Зато MapReduce-подход масштабируется на объёмы, которые не помещаются в память одной машины (каждый маппер работает со своим чанком, а память определяется размером чанка и числом ключей в партиции), а SQL-запрос с самосоединением на сотни миллионов вакансий в памяти не поместился бы.

9. Выводы

- Хранилище RustFS поднято, данные и результаты лежат в бакете `hh-lake`; MapReduce читает файл из хранилища по частям и переживает падение маппера.
- Два независимых способа расчёта дали **полностью одинаковые** результаты для 97 навыков, 4 574 пар и 82 364 троек.
- Число пар и троек растёт комбинаторно: C(n, 2) и C(n, 3) на вакансию, поэтому троек почти в 18 раз больше, чем пар.
- Зарплату повышают навыки, связанные с низкими задержками и торговыми системами (Low latency, алготрейдинг, SIMD, C++20), понижают — устаревший десктопный стек (MFC, Delphi, Pascal, SVN) и «школьные» требования (ООП, алгоритмы). Часть «падений» пар (C++20 + Redis) объясняется составом вакансий, а не самим навыком.
- Регионы различаются и уровнем зарплат (Москва ≈ 1.6 раза выше Новосибирска), и профилем рынка: финансы, игры, встраиваемые системы.
- Оптимизация MapReduce ускорила расчёт в 2.2 раза ценой памяти; в один процесс он быстрее и экономнее по памяти, чем первая версия. DuckDB быстрее всех, но самый затратный по памяти.

Ограничения. Датасет синтетический (генератор `gen_data.py`), поэтому выводы о зарплатах и регионах описывают этот датасет, а не реальный рынок. Курсы валют и

НДФЛ заданы константами. Время и память зависят от машины, на другом компьютере абсолютные числа будут другими, соотношение между способами должно сохраниться.